

BACKEND DEVELOPER PORTFOLIO

최시원

Backend Developer

AI 경험 제외

JAVA SPRING MYSQL POSTGRESQL REDIS

PROJECT SUMMARY

01 / AUTHORIZATION

HDP ERP

User Service 원본과 Redis UserSnapshot으로 권한을 조회했습니다. 정보 변경은 캐시 무효화로 반영하고 삭제 실패는 Outbox로 재처리했습니다.

02 / LOCATION SEARCH

ToiletNearBy

조회 범위와 캐시 대상을 조정하고, k6 부하 시험으로 위치 검색 응답 시간을 비교했습니다.

03 / MATCH QUERY

GAME_MATCH

실제 목록 API의 반복 SQL을 계측하고 CTE 조회안과 응답을 자동 대조했습니다. 동시 승인 전략은 별도 실험으로 비교했습니다.

상세본 구성

ERP는 담당 기능을 먼저 설명하고, 로그인 동시성과 인가 정보 갱신 문제를 이어서 다룹니다. 다른 프로젝트는 조회와 동시 승인 사례에 집중했습니다.

01 **HDP ERP** P. 3–15

UserSnapshot 인가와 변경 반영

서비스 소개 → 계정과 인증, 경량 프레임워크와 Outbox 구조 → 동시성 해결과 조회 성과

02 **ToiletNearBy** P. 16–19

DB 조회와 후보 캐시

서비스 소개 → 범위 조회 → 후보 캐시 → 검색 응답 개선

03 **GAME_MATCH** P. 20–23

쿼리와 승인 트랜잭션

서비스 소개 → N+1, EntityGraph와 CTE → 락 비교와 선택

04 **추가 프로젝트** P. 24

그 밖의 개발 경험

웹 서비스, 데이터베이스, IoT

자동차 부품 주문과 재고 관리

가상 자동차 부품 회사 현대파츠의 지점과 본사가 사용하는 ERP입니다. 고객 주문부터 재고 이동, 구매 발주와 생산 작업까지 5개 서비스로 나눴습니다.

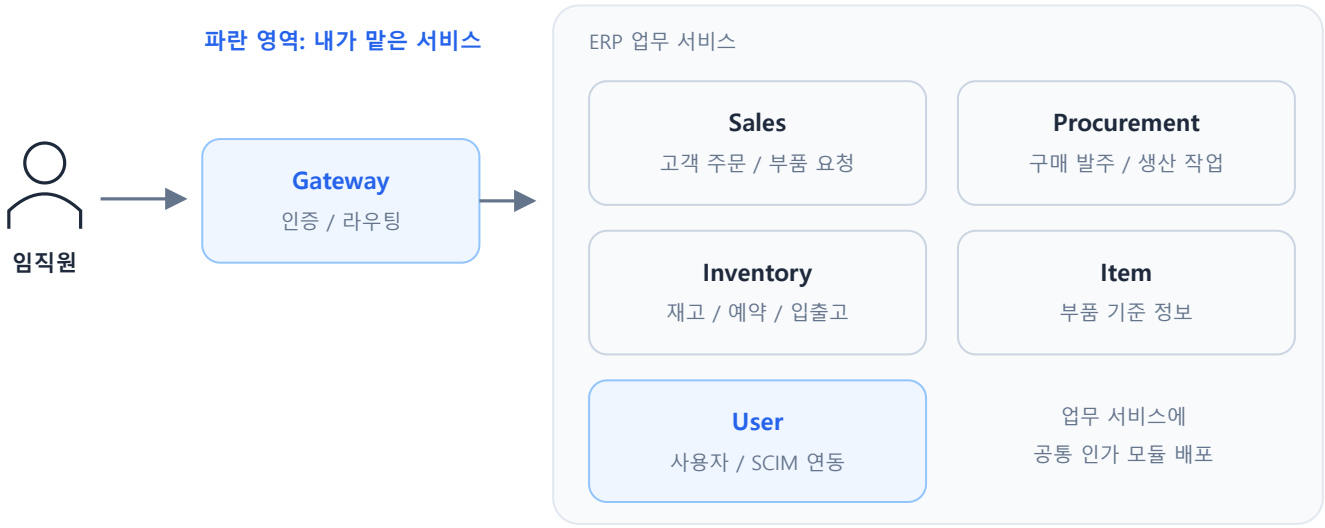
담당 5인 팀에서 User Service, Gateway와 경량 프레임워크를 맡았습니다. 로그인 연동, 사용자 정보 동기화, 업무 API의 권한 검증을 구현했습니다.

서비스 이용 흐름

고객 주문 → 지점 재고 확인 → 본사에 부품 요청 → 부품 확보와 이동

지점에 재고가 있으면 바로 출고합니다. 본사 재고도 부족하면 구매 또는 생산 작업을 진행합니다.

서비스 구성과 담당 영역



구현 범위 이 가운데 계정 연동과 인증, 업무 권한을 맡았습니다. 다음 장부터 공통 로그인과 ERP 사용자 정보를 나눈 구조, 권한 조회와 변경 반영 과정을 설명합니다.

이어서 ERP 계정 연동

계정 생명주기와 ERP 연동

HDP 인사 시스템을 출발점으로 공통 로그인 계정과 ERP 사용자 정보를 각각 생성하고 변경하는 구조입니다.

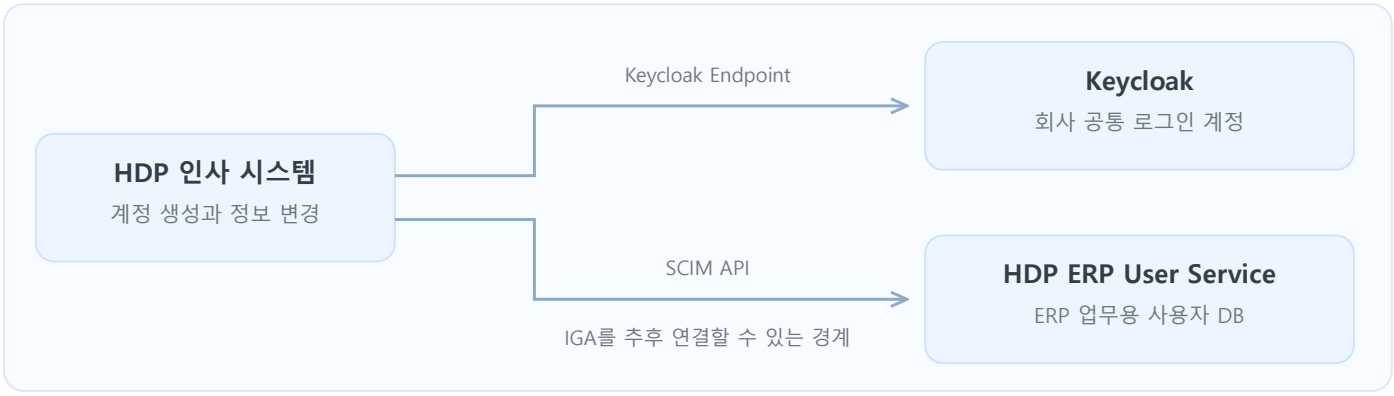
구조

IGA와 AD/LDAP를 통한 전사 프로비저닝을 검토했지만 프로젝트 범위에서는 제외했습니다. 인사 시스템에서 Keycloak Endpoint와 ERP의 SCIM API로 전달되는 두 경로를 분리했습니다.

계정 생명주기와 최종 적용 범위



최종 적용: IGA와 AD/LDAP를 생략하고 연동 경계를 분리



결과

ERP의 계정 생성, 변경과 비활성화를 SCIM API로 받도록 구성했습니다. 외부 IGA를 나중에 연결해도 ERP 사용자 관리의 경계는 유지하도록 설계했습니다.

조건과 범위

상단은 별도 검증 범위이며 최종 ERP 연동에 포함되지 않습니다. 회사 전체 IGA를 구축한 것은 아니며, ERP 측 SCIM API와 Keycloak 로그인 연동을 구현했습니다.

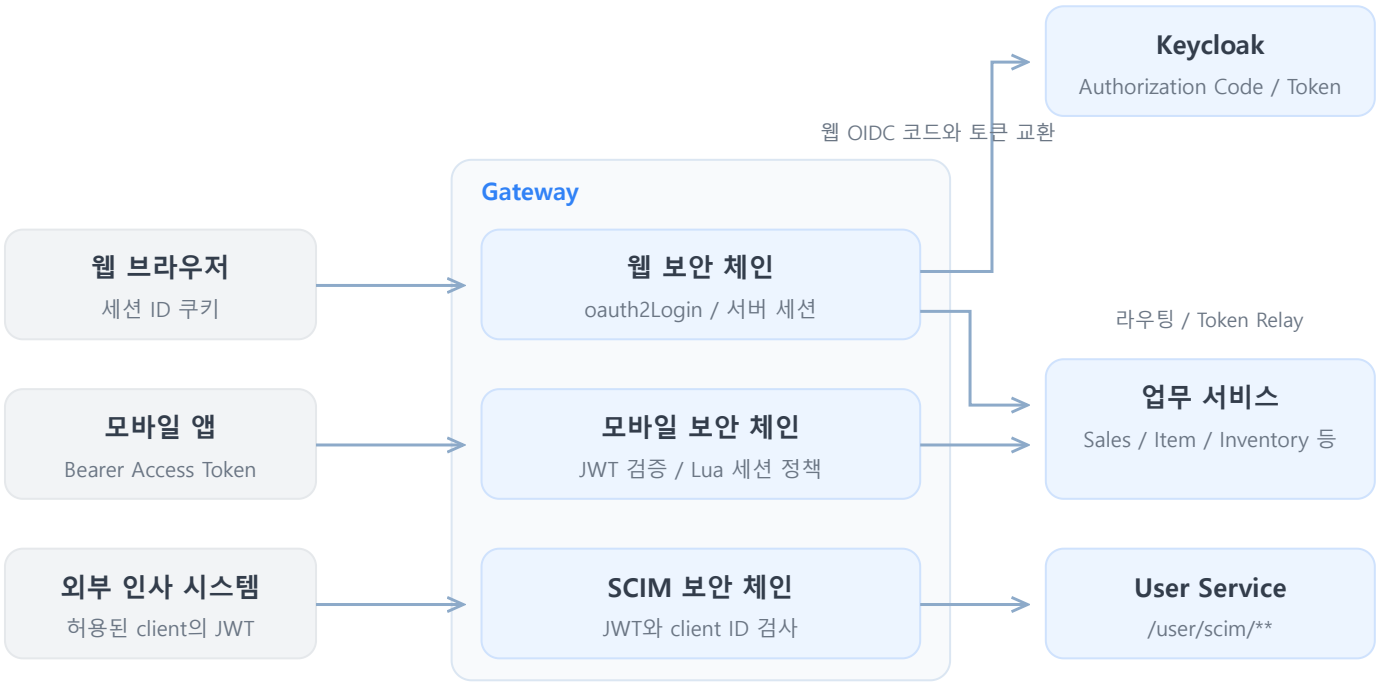
Gateway의 인증과 라우팅

외부 HTTP 요청을 받아 User, Sales, Procurement 등 ERP 서비스로 전달하는 진입점을 구현했습니다.

책임

웹, 모바일과 SCIM 요청은 서로 다른 보안 체인으로 처리했습니다. 공통 인증은 Gateway, 업무별 역할 판단은 각 서비스의 경량 프레임워크가 맡습니다.

Gateway의 요청 분리와 서비스 라우팅



구현

요청 유형별 정책

웹은 OIDC 로그인과 세션, 모바일은 JWT 검증을 적용했습니다. SCIM 요청은 별도 체인에서 JWT와 허용된 클라이언트인지 확인했습니다.

경로 기반 전달과 Token Relay

/sales/**는 Sales, /user/**는 User Service로 연결했습니다. 업무 서비스가 인증 정보를 사용할 수 있도록 필요한 경로에 Token Relay를 설정했습니다.

결과

인증, 보안 정책과 라우팅을 공통 진입점에 모으고, 각 서비스는 자신의 업무 로직과 역할 규칙을 담당하도록 나눴습니다.

웹 세션과 모바일 토큰 인증

Gateway에서 웹, 모바일, 외부 SCIM 요청을 서로 다른 보안 체인으로 처리했습니다.

문제

웹은 Gateway가 로그인 흐름을 진행하지만, 앱은 AppAuth로 Keycloak과 직접 로그인합니다. 토큰을 받는 위치에 맞춰 요청 인증 방식을 구분했습니다.

웹 요청

브라우저	→	Gateway	→	업무 서비스
세션 ID 쿠키 전달		서버 로그인 상태 확인		요청 전달

모바일 요청

모바일 앱	→	Gateway	→	업무 서비스
Bearer JWT 전달		JWT와 sid 검증		요청 전달

구현

외부 시스템 요청은 별도 제한

SCIM API는 JWT 검증뿐 아니라 허용한 클라이언트인지도 확인했습니다. 일반 사용자와 인사 시스템의 접근 조건을 구분했습니다.

결과

로그인은 같은 Keycloak을 사용하면서 채널별 인증 상태와 외부 연동 정책을 Gateway에서 처리했습니다.

Keycloak 로그아웃 연동

ERP의 인증 상태와 Keycloak 로그인 세션을 함께 정리했습니다.

문제

ERP 세션이나 앱의 토큰만 삭제하면 Keycloak 세션은 남았습니다. 로그아웃 후 로그인 버튼을 다시 눌렀을 때 기존 인증을 재사용하는 문제가 있어 인증 서버에도 종료를 요청했습니다.

로그아웃 처리

로그아웃 요청	→	앱 상태 정리	→	Keycloak 요청	→	로그인 세션 종료
웹 또는 앱		세션 또는 토큰 제거		end_session_endpoint		Keycloak에서 처리

클라이언트별 구현 순서는 다르며, 두 종류의 인증 상태를 모두 정리하는 흐름입니다.

구현

웹

Spring Security의 OIDC 로그아웃 핸들러를 연결했습니다. Gateway 세션 정리와 별도로 Keycloak 세션 종료를 요청합니다.

모바일

앱의 토큰과 Gateway에서 관리하는 로그인 상태를 정리하고, Keycloak의 세션 종료 요청을 진행했습니다.

결과

로그아웃 뒤 로그인 과정에서 남아 있던 인증 세션을 재사용하는 문제를 수정했습니다.

웹과 모바일의 중복 로그인 정책

사용자 식별은 sub로 통일하고, 웹은 기존 세션을 만료시키며 모바일은 다른 로그인 세션의 요청을 차단했습니다.

문제 웹 세션과 모바일 JWT는 상태를 보관하는 방식이 달라 채널별로 중복 접근 정책을 나눴습니다.

같은 사용자, 서로 다른 중복 로그인 정책

웹: 새 로그인 허용

사용자 sub → 웹 세션 1개

↓

새 로그인 → 기존 웹 세션 만료

Spring Session / maximumSessions(1)

모바일: 기존 세션 유지

사용자 sub → 현재 sid 1개

↓

다른 sid 요청 → 차단

Redis / Lua 비교와 조건부 등록

웹: 신규 로그인은 허용, 기존 세션 만료

```
.maximumSessions(1)
.maxSessionsPreventsLogin(false)
.expiredSessionStrategy(sessionExpiredStrategy)
.sessionRegistry(sessionRegistry)
```

SecurityConfig.java / 실제 설정 발췌

웹: 동일 사용자를 sub로 묶기

```
@Override
protected String name(Object principal) {
    return subjectExtractor.extract(principal)
        .orElseGet(() -> super.name(principal));
}
```

SpringSessionBackedSessionRegistry.name() / 발췌

결과

웹은 새 로그인을 우선하고, 모바일은 Redis에 등록된 sid를 유지합니다. 같은 사용자가 웹과 모바일을 각각 사용하는 것은 허용합니다.

조건과 범위 모바일의 sid는 기기 고유 번호가 아니라 Keycloak 로그인 세션 ID입니다. 다음 장에서 모바일의 조회와 등록을 Lua로 묶은 코드를 설명합니다.

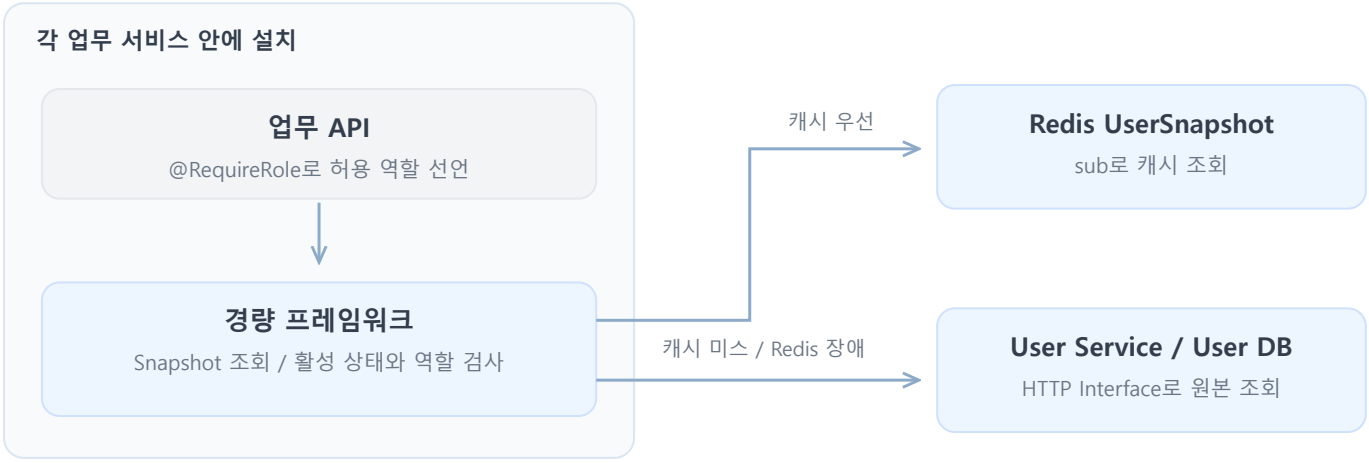
경량 프레임워크와 사용자 정보 조회

각 업무 서비스에 설치한 경량 프레임워크가 UserSnapshot을 조회하고 상태와 역할을 검증합니다.

문제

서비스마다 원본 조회와 권한 검사를 반복하지 않도록 조회 경로를 묶었습니다. User Service가 정보를 모든 서비스에 밀어주는 대신, 각 서비스의 경량 프레임워크가 필요할 때 조회합니다.

업무 서비스 내부의 경량 프레임워크



조회한 Snapshot으로 상태와 역할을 확인한 뒤 업무 API 실행을 허용합니다.

별도 인가 서버가 아니라 각 서비스의 라이브러리입니다. 원본 조회 결과는 Snapshot으로 반환하고 캐시에 저장합니다.

구현

패키지 배포와 적용 가이드

GitHub Packages에 배포하고 의존성, 연결 설정, 애노테이션 사용법을 정리했습니다. 팀원이 업무 API에 모듈을 적용하고 시험할 수 있도록 지원했습니다.

자동 설정 순서 수정

Redis 연결 Bean보다 보안 설정이 먼저 평가돼 캐시 Bean이 등록되지 않는 문제를 수정했습니다. Redis 자동 설정 이후 보안 설정을 실행하도록 순서를 지정했습니다.

```
@AutoConfiguration(after = DataRedisAutoConfiguration.class)
```

결과

업무 서비스는 공통 구현을 가져다 쓰고, API마다 허용할 역할을 선언하는 방식으로 인가를 적용했습니다.

문제 상황

토큰 권한 지연과 원본 조회 비용

→

해결 방법

UserSnapshot 기반 인가

ERP의 업무 권한은 현재 인사 정보로 결정돼야 했습니다. 토큰에 남은 역할을 사용하지 않고 User Service를 기준으로 인가 정보를 조회했습니다.

문제

JWT는 만료 전 인사 변경을 반영하지 못하고, 매 요청 원본 조회는 네트워크와 DB 비용이 반복됐습니다. User DB를 원본으로 두고 Redis UserSnapshot으로 조회 경로를 단축했습니다.

공통 모듈의 사용자 정보 조회

사용자 식별
인증 정보의 sub 사용

→

Redis 조회
UserSnapshot 우선 확인

→

권한 검사
상태와 역할 판단

캐시가 없거나 Redis 조회가 실패하면 User Service의 원본을 조회합니다.

구현

조회 최적화와 원본 유지

Redis가 적중하면 원격 호출을 생략합니다. 캐시가 없거나 Redis에 장애가 있으면 HTTP Interface로 User Service를 조회합니다. 원본도 확인할 수 없으면 접근을 허용하지 않습니다.

변경 시 캐시 삭제

SCIM으로 사용자 정보가 갱신되면 Snapshot을 삭제합니다. 다음 인가 요청이 변경된 원본을 다시 읽도록 했으며, 삭제 실패는 Outbox로 재처리했습니다.

결과

Redis 중단 중에도 실제 User Service로 조회가 전환됐습니다. 원본 서비스까지 중단하면 검증 진입점은 503을 반환해 접근을 허용하지 않았습니다.

조건과 범위

원본 호출의 연결 1초, 응답 3초 제한이 있습니다. Redis 장애 시 원본 부하를 제한하는 별도 circuit breaker나 bulkhead는 이번 코드에서 확인하지 못했습니다.

이어서 사용자 변경 반영

문제 상황

인사 변경 후 이전 캐시 잔존

→ 해결 방법

변경 시 캐시 무효화

인사 정보 변경이 SCIM API로 들어오면 User DB를 갱신하고 기존 UserSnapshot을 삭제합니다.

문제

DB만 갱신하고 Redis에 이전 정보가 남으면 이후 요청이 예전 권한으로 처리될 수 있었습니다. 변경된 정보를 다음 조회에 반영하려면 기존 캐시를 지워야 했습니다.

변경 반영 순서

SCIM 요청	→	DB 저장	→	캐시 삭제	→	다음 인가 요청
사용자 정보 변경		원본 갱신		기존 Snapshot 제거		원본을 다시 조회

구현

변경 시 삭제

TTL이 끝날 때까지 기다리는 대신 커밋 후 캐시를 삭제했습니다. 다음 요청은 User Service에서 변경된 정보를 가져옵니다.

삭제 실패 기록

Redis 삭제는 DB 저장과 별도로 실패할 수 있습니다. 삭제 대상은 DB 변경과 함께 Outbox에 저장해 이후에도 재시도할 수 있도록 했습니다.

결과

실제 사용자 변경 후 Snapshot 삭제와 Outbox DONE을 확인했습니다. 다만 변경 전 조회가 늦게 끝나면 삭제 뒤 과거 Snapshot이 다시 저장되는 경쟁도 재현됐습니다.

조건과 범위

Snapshot TTL은 300초입니다. 구버전 재저장을 막는 version 비교나 CAS 저장은 미구현입니다. 무효화만으로 모든 요청의 최신 권한을 보장한다고 표현하지 않았습니다.

이어서 Outbox 재시도

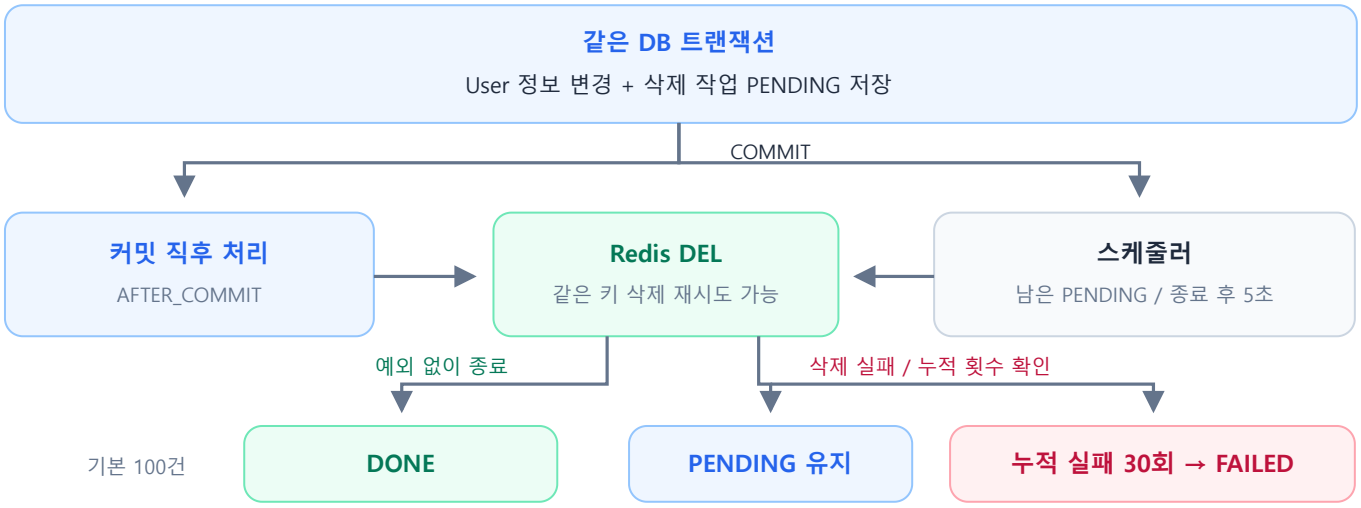
캐시 삭제와 Outbox 재시도 구조

User Service가 DB 변경 후 Redis의 사용자 캐시를 삭제하는 과정입니다.

문제

DB 저장 후 Redis가 실패하거나 프로세스가 종료되면 캐시 삭제를 끝내지 못할 수 있었습니다. 사용자 정보와 삭제할 작업을 같은 트랜잭션에 저장해 미완료 작업을 추적했습니다.

커밋 이후 삭제와 남은 작업 재처리



구현

사용자 변경과 삭제 작업을 함께 저장

사용자 정보와 캐시 삭제 대상을 같은 DB 트랜잭션에 기록합니다. 둘 중 하나가 저장되지 않으면 전체를 롤백하고, 커밋 후 Redis 삭제를 시도합니다.

미완료 작업만 주기적으로 재시도

스케줄러는 PENDING 작업을 조회해 삭제를 다시 시도합니다. 성공하면 DONE, 누적 실패가 상한에 도달하면 FAILED로 전환하며, FAILED는 자동 재시도 대상에서 제외합니다.

결과

Redis 중단 중에는 작업이 PENDING으로 남고, Redis 복구 후 DONE이 되는 것을 확인했습니다. Outbox 저장 실패 시 사용자 변경도 함께 롤백됐습니다.

조건과 범위

격리 PostgreSQL 17, Redis 7, Keycloak 26과 실제 User Service로 검증했습니다. 시험용 1초 / 3회 설정에서는 FAILED로 멈췄으며 자동 재처리는 없습니다. 기본값은 5초 / 30회입니다.

문제 상황

동시 요청의 중복 허용

→

해결 방법

Lua 원자 처리

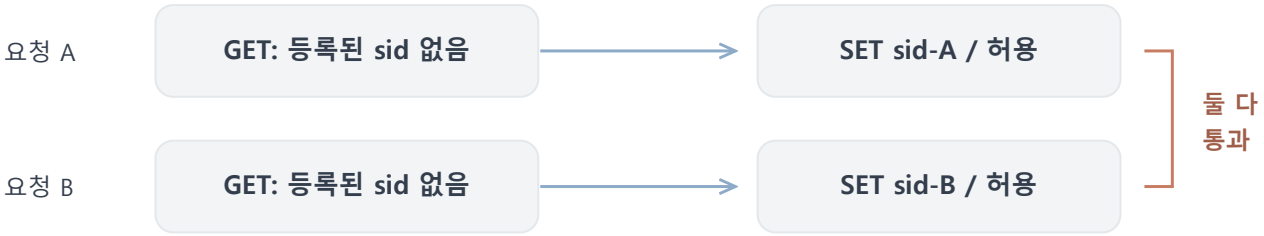
사용자당 하나의 모바일 로그인 세션을 유지하기 위해, sid 조회와 조건부 등록 사이의 경쟁을 막았습니다.

문제

GET과 SET이 별도 명령이면 A와 B가 모두 빈 값을 읽어 통과할 수 있습니다. 같은 사용자라는 이유만으로 동시 요청이 순서대로 처리되지는 않습니다.

문제: GET과 SET 사이에 다른 요청이 끼어드는 경우

동일 사용자 sub, 서로 다른 sid의 요청 A와 B



A가 저장하기 전에 B도 빈 값을 읽으면, 사용자당 한 세션 정책을 위반합니다.

구현

해결: 조회, 비교와 등록을 한 번에 실행

A의 Lua 실행이 끝난 뒤 B가 실행됩니다. A가 sid-A를 등록했다면 B는 빈 값이 아닌 sid-A를 읽고, 다른 sid-B의 요청을 차단합니다.

sid 비교: 같은 세션만 재사용

```
local curSid = string.sub(cur, 1, i - 1)
if curSid == sid then
    take = true
end
```

RedisMobileSessionStore.java / 실제 분기 발췌

조건부 저장: 허용이면 등록, 아니면 차단

```
if take then
    redis.call('SET', KEYS[1],
        sid .. '|' .. ARGV[2], 'EX', ttl)
    return 1
end
return 0
```

같은 REGISTER_OR_VALIDATE 스크립트 / 발췌

결과

서로 다른 sid가 각각 20회 요청한 시험에서 한 sid의 20건만 허용되고 다른 20건은 거절됐습니다. 같은 sid의 반복 요청은 허용하며 TTL을 갱신합니다.

조건과 범위

코드는 전체 스크립트의 핵심 분기 발췌입니다. 앞부분에서 GET, 빈 값과 저장 형식을 검사합니다. 두 발췌는 같은 Lua 안에서 실행됩니다. 실제 Redis 스크립트 수준 검증이며 Gateway 전체 시험은 아닙니다.

문제 상황

삭제 작업 경합과 기록 실패

→

해결 방법

잠금 분담과 재시도

여러 서버가 미완료 캐시 삭제 작업을 나눠 처리하고, 삭제 후 완료 기록에 실패한 작업도 다시 처리하도록 구성했습니다.

문제

여러 서버가 같은 삭제 작업을 동시에 가져갈 수 있습니다. 또 캐시는 삭제됐지만 DB에 완료 상태가 저장되지 않으면, 다음 실행에서 같은 작업을 다시 처리하게 됩니다.

캐시 삭제 작업 처리 순서

미완료 작업 조회	→	다른 작업 선택	→	캐시 삭제	→	완료 기록
처리할 DB 행 잠금		다른 서버가 잠근 행 제외		Redis DEL 실행		DB에 DONE 저장

구현

다른 서버가 처리 중인 작업은 건너뛰기

FOR UPDATE SKIP LOCKED로 처리할 행을 잠급니다. 격리 PostgreSQL의 동시 트랜잭션 두 개가 PENDING 4건을 2건씩 겹치지 않게 가져가는 것을 확인했습니다.

삭제 후 완료 기록이 실패해도 재처리

격리 DB에서 완료 저장을 강제로 실패시켰습니다. 캐시는 이미 삭제됐지만 작업은 PENDING으로 남았고, 다음 실행에서 없는 키를 다시 삭제해도 오류 없이 DONE으로 완료됐습니다.

결과

처리 중인 작업은 잠금으로 구분하고, 완료 기록이 남지 않은 작업은 재시도했습니다. Redis DEL은 같은 키에 반복 실행해도 안전해 삭제와 DB 기록 사이의 실패를 복구할 수 있었습니다.

조건과 범위

행 분배는 두 DB 트랜잭션으로 검증했습니다. 두 User Service 인스턴스 전체나 프로세스 강제 종료를 시험한 것은 아닙니다. 원본 코드 대신 격리 DB에만 장애 트리거를 적용했습니다.

문제 상황

반복 HTTP와 DB 조회

→

해결 방법

Redis Snapshot 재사용

경량 프레임워크가 UserSnapshot을 재사용해 매번 User Service와 DB를 조회하는 비용을 줄였습니다.

문제

토큰의 오래된 권한 대신 현재 사용자 정보를 읽으려면 원본 조회가 필요했습니다. 반복되는 HTTP 호출과 DB 접근을 줄이기 위해 Redis에 인가용 Snapshot을 두었습니다.

구현

실제 조회 경로를 같은 조건으로 비교

검증용 HTTP 진입점에서 실제 core와 User Service, PostgreSQL 17을 사용했습니다. 같은 활성 사용자로 직접 조회와 사전 적재한 캐시를 비교했고, HTTP 실패는 두 경로 모두 0건이었습니다.

결과

Snapshot 조회 p95

60.89 → 32.81ms

캐시 적중 시 원본 호출

0회

직접 조회는 2932회

이번 단일 시험에서 캐시 적중 경로의 p95가 약 46% 낮았고, 원본 호출을 생략하는 효과를 확인했습니다.

성과를 확인한 부하 결과

100RPS, 30초	원본 직접 조회	Redis 적중
p50 / p99	12.41 / 3075.58ms	5.42 / 74.06ms
실효 RPS / 미시작 반복	97.73 / 68건	100.03 / 0건
HTTP 실패	0건	0건

조건과 범위 2026-10-31, k6 0.55, 100RPS 30초, VU 50~200. 동일 사용자 1명과 로컬 Docker 통신 조건입니다. Gateway와 ERP 업무 Controller, @RequireRole AOP를 포함한 전체 API 성능은 아닙니다.

주변 화장실 검색과 후기 공유

현재 위치나 검색한 장소 주변의 공중화장실을 찾는 웹 서비스입니다. 상세 정보와 후기를 확인하고 별점, 비밀번호 등 이용 정보를 공유할 수 있습니다.

3인 팀 백엔드 개발 / 주변 검색, 장소 검색 연동과 JWT 인증 / 이후 개인 리팩터링

주변 검색의 처리 범위



초록 점은 결과에 포함, 모서리의 빨간 점은 제외
계정 인증이 필요한 후기와 별점은 별도 API로 처리

주요 기능	서비스 설명
주변 화장실 검색	현재 위치와 반경을 기준으로 공중화장실을 찾고 상세 정보 확인
장소 검색	Kakao Local API로 장소를 검색하고 해당 위치 주변의 화장실 탐색
후기와 이용 정보	후기, 별점과 비밀번호 등 이용 정보를 작성하고 공유
사용자 인증	JWT 기반 로그인과 사용자 식별, 작성 기능의 접근 제어

개선 대상 이 가운데 주변 화장실 검색의 반복 조회 비용을 줄였습니다. DB 조회 범위를 축소한 뒤 격자별 후보 캐시를 적용하고, 실제 위치와 반경은 요청마다 다시 계산했습니다.

이어서 좌표 범위 조회

문제 상황

검색마다 전체 데이터 조회

→

해결 방법

좌표 범위와 복합 인덱스

위치 검색 API가 사용자 주변의 화장실 후보만 조회하도록 변경했습니다.

문제 원형 반경 안의 데이터를 찾기 위해 모든 좌표의 거리를 계산하고 있었습니다. DB에서는 좌표 범위를 먼저 줄이고, 애플리케이션에서 정확한 반경을 판정하도록 나눴습니다.

DB 후보 범위와 실제 검색 결과



초록 점은 결과에 포함, 모서리의 빨간 점은 제외
계정 인증이 필요한 후기와 별점은 별도 API로 처리

결과

검색 결과를 유지하면서 DB 후보 수를 줄였습니다. 사각 범위 밖 데이터의 전송과 거리 계산을 생략했습니다.

세부 내용

측정 항목	전체 조회	범위 조회
조회 후보	100000건	1144건
DB 실행 시간	21.19ms	0.342ms
최종 결과	1000건	1000건

조건과 범위 Docker PostgreSQL 17, 동일 좌표와 반경. DB 실행 시간은 EXPLAIN ANALYZE 기준입니다.

이어서 격자 후보 캐시

문제 상황

좌표별 캐시 분산

→ 해결 방법

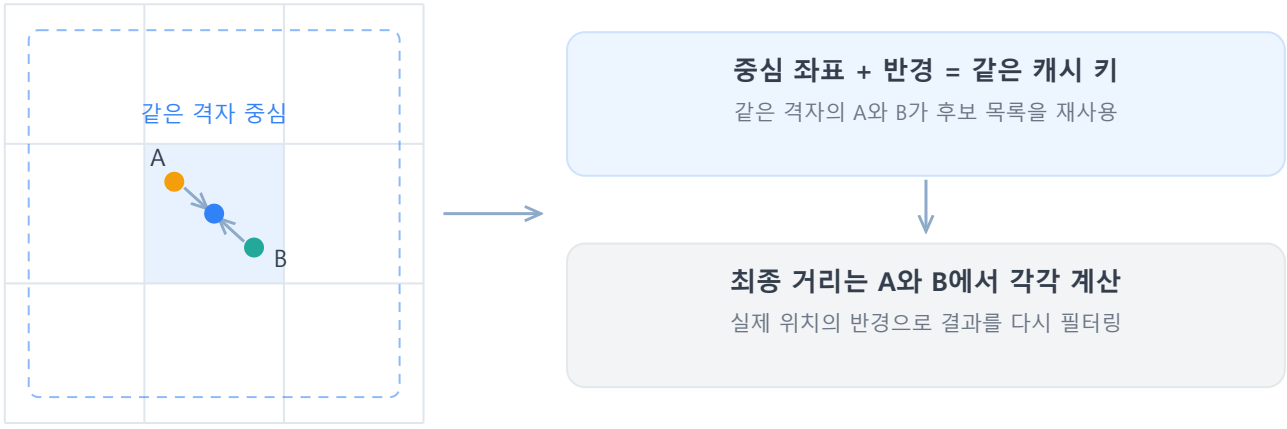
격자 중심 정규화

위도와 경도를 격자 중심으로 정규화하고, 중심 좌표와 반경을 캐시 키로 사용했습니다.

문제

최종 결과는 실제 위치에 따라 달라집니다. 후보만 공유하고 거리와 반경은 요청마다 계산하며, 중심 이동으로 후보가 빠지지 않게 조회 범위를 확장했습니다.

격자 중심 정규화와 요청별 거리 계산



파선: 중심 이동을 보완한 후보 조회 범위

격자 중심은 후보 조회에만 사용합니다. 도식은 비례를 단순화했으며 최종 검색 위치는 A와 B 그대로입니다.

구현

확장 범위도 비교

0.005도 격자에서 800m와 370m 모두 68좌표의 결과가 일치했습니다. cold 이동 요청에서 후보 평균은 304.6개에서 253.8개, p95는 5.77ms에서 4.1ms로 줄었습니다.

전체 탐색과 자동 대조

경계와 모서리를 포함한 68좌표에서 ID 집합과 반올림 거리의 차이는 0건이었습니다. 대표 좌표와 300m, 1km, 3km를 조합한 9조건도 일치했습니다.

결과

검색 결과를 유지하면서 후보를 줄일 가능성을 확인했습니다. 대표 조건만으로 최적값을 단정하지 않고 제품의 800m 설정은 유지했습니다.

조건과 범위

로컬 PostgreSQL 17.5, 합성 10만 건. 비교 부하는 별도 HTTP 클라이언트로 각 조건 100RPS, 4초, 400요청을 실행했습니다. ID 집합과 반올림 거리를 대조했으며 거리순 정렬은 보장하지 않습니다.

문제 상황

반복 후보 조회와 응답 지연

→ 해결 방법

후보 캐시 재사용

격자 후보 캐시로 반복 DB 조회를 줄여 주변 검색의 응답 지연을 개선했습니다.

문제

좌표 범위를 줄여도 요청마다 비슷한 후보를 DB에서 다시 읽었습니다. 재사용 가능한 후보 조회와 사용자 위치마다 달라지는 거리 계산을 분리했습니다.

구현

같은 데이터와 요청 조건

PostgreSQL 17.5에 합성 데이터 10만 건을 적재하고, 고정 좌표와 반경 3km로 120초간 실행했습니다. 각 경로를 워밍업하고 캐시 경로는 후보를 한 번 적재한 뒤 측정했으며 결과는 모두 1000건이었습니다.

결과

검색 응답 p95

41.57 → 8.52ms

p95 감소

79.5%

이 로컬 단일 시험에서 후보 캐시 경로의 p95가 약 79.5% 낮았고, 시작하지 못한 반복 없이 목표 100RPS에 가까운 처리량을 확인했습니다.

같은 검색 결과를 반환한 부하 비교

측정 항목	범위 조회	후보 캐시
평균 응답	16.87ms	4.47ms
실제 처리량	99.342RPS	99.992RPS
미시작 반복 / HTTP 실패	76건 / 0%	0건 / 0%

조건과 범위

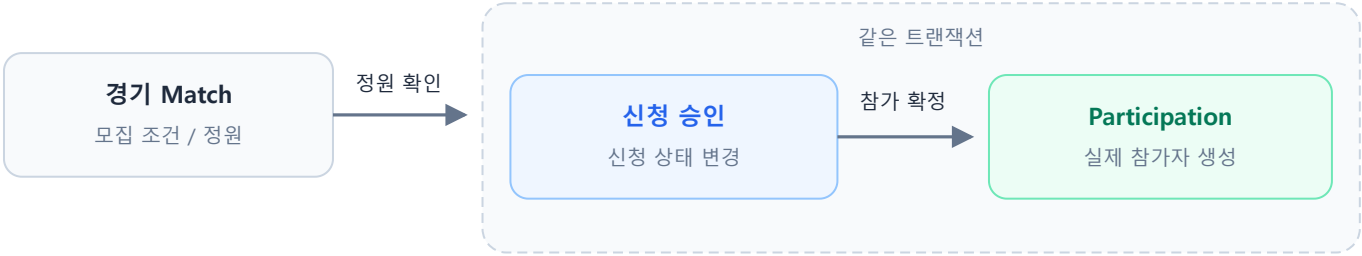
2026-10-31, k6 v0.50.0, VU 30~200. dropped_iterations는 서버 오류가 아니라 시작하지 못한 반복입니다. 이 고정 좌표의 단일 실행 결과이며 운영 성능을 보장하지 않습니다.

온라인 게임 친선경기 매칭

게임 이용자가 친선경기를 만들고 참가를 신청하는 커뮤니티 서비스입니다. 게임별 프로필과 경기 모집, 게시판, 댓글, 친구 기능을 제공합니다.

1인 백엔드 개발 / 게임 프로필, 친선경기와 커뮤니티 API / 이후 조회와 동시성 리팩터링

신청에서 참가 확정까지



팀 신청은 신청 건수와 실제 참가 인원을 구분

주요 기능	서비스 설명
게임 프로필	게임별 사용자 프로필을 등록하고 친선경기 활동에 사용
친선경기 모집	게임과 일정, 모집 조건을 등록하고 경기 목록과 상세 정보 조회
신청과 참가 관리	참가 신청과 승인, 거절, 취소를 처리하고 승인된 참가자 관리
커뮤니티	게시글과 댓글 작성, 친구 관계 등 게임 이용자 간 소통 기능

개선 대상

목록 조회에서는 N+1을 EntityGraph로 줄인 뒤 남은 집계를 CTE로 구성했습니다. 참가 승인에서는 정원 초과를 막는 두 락 방식의 비용을 비교하고 짧은 승인 트랜잭션과 재시도 비용을 고려해 비관적 락을 선택했습니다.

이어서 경기 목록 조회 개선

문제 상황

N+1과 경기별 반복 집계

→

해결 방법

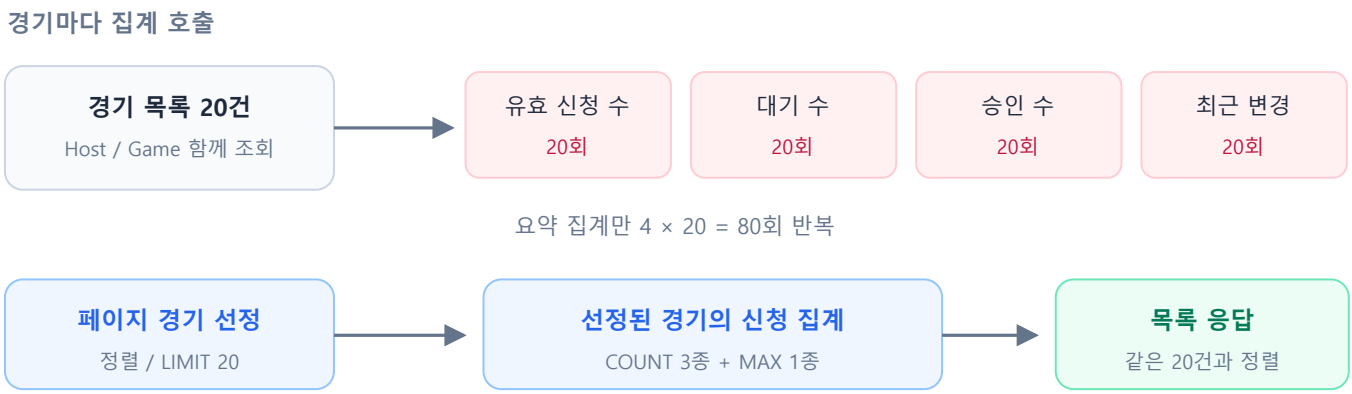
EntityGraph에서 CTE로

주최자와 게임 연관 조회를 먼저 합치고, 남은 신청 집계를 페이지 단위로 구성했습니다.

문제

목록을 응답으로 바꾸며 지연 로딩과 명시적인 집계 호출이 반복됐습니다. 연관 조회 N+1과 경기별 COUNT / MAX 호출은 발생 원인이 달라 따로 다뤘습니다.

경기별 집계에서 페이지 단위 집계로



구현

EntityGraph로 연관 조회 축소

주최자와 게임을 목록과 함께 읽도록 지정했습니다. 별도 SQL 재현에서는 연관 조회 40회가 줄어 121회에서 81회가 됐지만, 경기 20건의 요약 집계 80회는 남았습니다.

페이지 선정 후 신청 집계

CTE에서 정렬된 경기 20건을 먼저 정하고 해당 경기의 신청만 집계했습니다. 유효 신청은 거절과 취소를 제외하고, 대기 수와 승인 수, 최근 변경 시각을 함께 반환했습니다.

결과

별도 SQL 재현 실험

121 → 81 → 1회

API와 CTE 자동 대조

20건 일치

연관 정보 결합 후에도 남는 반복 집계를 일괄 조회안으로 구성했습니다. 동일 fixture에서 ID, 순서, 경기 정보와 집계값, 최근 변경 시각의 차이는 0건이었습니다.

조건과 범위

실제 레거시 API는 EntityGraph 목록 1회 + 상태 확인 20회 + 집계 80회로 안정 상태 101회, 첫 상태 갱신 두 건 포함 시 103회였습니다. 위 121 → 81 → 1과 측정 범위가 다릅니다. CTE 적용 후 Java API 전체 계측은 미완료입니다.

문제 상황

동시 승인 시 정원 초과

→

해결 방법

HTTP 승인 경로 비교

평소에는 순차 승인하고 드물게 두 관리자가 같은 경기를 동시에 변경하는 상황을 시험했습니다.

문제

정원이 넉넉한 일반 승인과 마지막 한 자리 경쟁은 구분해야 합니다. 실패 응답이 빠른 것만으로 성능을 평가하지 않고, 완료 결과와 재시도 횟수를 함께 확인했습니다.

구현

일반 승인과 마지막 한 자리 경쟁

정원 100명에서 100건을 승인하며 동시 요청 쌍을 0, 1, 5개씩 배치하고 각 조건을 3회 반복했습니다. 별도로 정원 10명 중 9명이 찬 경기에서 두 요청을 20회 경쟁시켰습니다.

측정 경계와 재시도

클라이언트 연결을 미리 열고 동시 시작 지점 이후 POST 전송부터 응답 본문 수신까지 perf_counter_ns로 측정했습니다. 낙관적 락은 충돌 시 새 트랜잭션에서 최대 1회 재시도하며, 이 시간도 포함했습니다.

결과

두 방식 모두 정합성을 지켰습니다. 이 시험에서는 비관적 락의 전체 평균이 낮았고, 마지막 한 자리 경쟁에서 충돌 후 재실행도 줄었습니다. 짧은 승인 작업에서는 대기 후 최신 상태로 처리하는 방식을 선택했습니다.

동일한 업무 처리로 비교

시험 조건	낙관적 락	비관적 락
일반 승인: 전략별 900요청	900건 승인, 버전 충돌 18건	900건 승인, 버전 재시도 없음
마지막 한 자리: 전략별 40요청	20건 승인, 20건 정원 마감, 60회 시도	20건 승인, 20건 정원 마감, 40회 시도
전체 HTTP 평균: 전략별 940요청	5.55ms	4.65ms
정원 초과 / 중복 참가	0건 / 0건	0건 / 0건

조건과 범위

로컬 MySQL 8.0.33과 테스트 전용 Spring HTTP 경로입니다. 제품 승인 API 전체의 결과가 아닙니다. 인위적 지연은 없으며, 과거 5ms 지연을 넣은 SQL 시험 수치는 선택 근거에서 제외했습니다. 작은 시간 차이로 일반적인 성능 우위를 단정하지 않습니다.

문제 상황

짧은 승인 작업의 충돌과 재시도

→ 해결 방법

비관적 락 선택

같은 경기의 승인 요청이 잠시 대기한 뒤 최신 정원을 확인하도록 설계했습니다.

문제

선택의 기준은 평균 응답 시간 하나가 아니었습니다. 정원 확인, 승인 상태 변경, 참가 정보 저장이 외부 호출 없이 짧은 트랜잭션으로 끝나고, 낙관적 락은 정원이 남아도 버전 충돌로 롤백과 재실행이 생길 수 있었습니다. 그래서 잠시 대기한 뒤 최신 정원으로 처리하는 비관적 락을 선택했습니다.

같은 경기의 승인 요청 처리 순서

시간 흐름 → / 같은 경기 행, 마지막 한 자리 예시



정원이 남아 있으면 B도 승인, 정원이 차면 마감 응답

구현

선택 기준: 정합성과 재실행 부담

경기 행을 잠근 뒤 정원과 신청 상태를 확인하고, 승인 상태와 참가 정보 및 인원을 하나의 트랜잭션에서 변경합니다. 충돌한 요청을 롤백하고 다시 실행하는 흐름보다, 최신 상태를 확인해 승인 또는 마감을 처리하는 흐름을 택했습니다.

감수하는 비용과 운영 조건

행 잠금 획득과 경합 시 대기는 감수합니다. 잠금 유지 시간을 줄이도록 외부 통신은 트랜잭션에 포함하지 않고, 잠금 타임아웃과 데드락도 별도 오류 처리 대상으로 봅니다.

결과

비관적 락은 짧은 승인 작업에서 정합성과 처리 흐름의 단순성을 위한 선택입니다. 별도 HTTP 시험의 전체 평균도 낙관적 락 5.55ms, 비관적 락 4.65ms로 나타났지만, 이 수치는 선택을 확인한 보조 근거로만 사용했습니다.

조건과 범위

승인 설계와 테스트 전용 HTTP 경로의 검증 결과입니다. 제품 코드의 승인 기능 전체가 반영됐다는 뜻은 아닙니다. 행 잠금은 신청 순서나 선착순 공정성까지 보장하지 않습니다.

Additional Projects

이외에 진행한 웹, 데이터베이스, IoT 프로젝트입니다.

FULL STACK 2026.03	**Recall 동창 검색, 재연결 서비스** 팀장, Frontend 담당	React, TypeScript 화면과 Spring REST API를 연동하고 AWS EC2, Vercel 배포를 구성했습니다. 팀원별 역할을 조율하며 서비스 흐름과 API 연결 기준을 정리했습니다.
FRONTEND 2026.02 - 2026.03	**개인 포트폴리오 웹사이트** 1인 개발	프로젝트 무한 Carousel과 관리 CRUD를 구현했습니다. 관계 데이터 삭제 순서를 직접 설계하며 화면 상태와 Database 무결성을 함께 고려했습니다.
WEB API 2025.07	**TikTok 광고 참여 설문 웹앱** 1인 개발	변경되는 설문 요구에 대응하도록 UI와 전송 로직을 분리했습니다. Google Apps Script Web App으로 응답 기록과 외부 API Key 은닉을 처리했습니다.
BACKEND 2023.09 - 2023.12	**약 복용 관리 및 보호자 알림 앱** 기획, Backend	사용자별 복약 일정과 복용 기록을 저장하는 API를 설계했습니다. 정해진 시간에 복용하지 않으면 보호자에게 Slack 알림을 전송하도록 구성했습니다.
DATABASE 2023.12	**병원 환자, 의료진 관리 웹 서비스** 1인 개발	환자, 간호사, 의사별 서비스를 분리하고 MySQL Schema를 정규화했습니다. Index 성능과 Transaction을 고려해 안정적인 데이터 구조를 설계했습니다.
IOT, MOBILE 2019.11 - 2019.12	**교내 화장실 사용 현황 애플리케이션** 팀장, 시스템 설계	초음파 Sensor로 칸별 사용 여부를 판별했습니다. 모바일 화면에서 빈 화장실을 실시간으로 확인할 수 있도록 Sensor와 Application을 연결했습니다.